

Adaptive Feature Selection for Predicting Video Quality Degradation

Machine Learning for Computer Systems · Project Proposal · Open Problem / Research

Aeliya Grover, Clarisse Cheung · September 2026

with help from Cursor and Claude

Project Summary

When you watch a streaming video, the player continuously chooses a *resolution* (how sharp the picture is). When the network slows down, *adaptive bitrate* logic responds by dropping to a lower resolution—a *downswitch*—to keep playback from stalling entirely. A downswitch is both a *quality-of-experience* (QoE) event users notice and the earliest visible sign that a session is in trouble.

Assignment 1 showed that *time-windowed network features*—simple statistics computed from packets over short time intervals, such as how many bytes arrived in the last second—can infer a session’s *current* resolution. Predicting an *upcoming* downswitch is both harder and more useful: a player or network that flags an at-risk session early can reroute traffic or adjust before quality drops.

A **feature** is one input number the model uses. Features differ enormously in what they cost to produce. Counting bytes requires only packet headers. Measuring round-trip time or retransmissions requires tracking *per-flow state*—remembering, for every connection, what has already been sent and acknowledged. Recovering video *chunk* boundaries (the individual pieces a video is downloaded in) requires inferring application structure inside an encrypted stream. These are not small differences, and a system that computes every feature on every session pays for all of them continuously.

Research question. Can *adaptive feature selection*—starting with cheap features and computing expensive ones only when needed—reduce the computational cost of predicting video quality degradation while matching the performance of a model that always uses the full feature set?

In plain terms: we want a warning system that says “this stream is about to degrade,” but we do not want to do expensive work on every session all the time. Easy cases should be cheap; hard cases can spend more compute.

This sits in the course’s suggested open-problem area of *the systems costs of different features*. It matters for real-time video analytics, where predictions must be made quickly and at scale.

Why downswitches and not stalls. We initially aimed to predict *rebuffering* (playback stalling outright). Inspecting the labels first, we found them too sparse to learn from: rebuffering appears in 2 of 1,000 sessions in the Netflix dataset (14 of 52,279 windows), and the multi-service dataset records no rebuffering at all. Downswitches capture the same underlying network stress at a learnable rate—the player lowers quality precisely to avoid a stall—and occur in 43% of sessions.

Data

We will use the course video QoE materials: `video_dataset.pkl` (204,713 ten-second windows across 4,000 sessions and four services—Netflix, YouTube, Twitch, and Amazon Prime Video) for modeling, and `netflix.pcap` for cost measurement. A `.pcap` file is a *packet capture*: a recording of the raw network traffic.

Every feature in this dataset is already tagged by *protocol layer*—how deep into the network stack you must look to compute it—which gives us our cost hierarchy directly:

- **L3** (11 features): throughput, byte and packet counts, parallel flows. Readable from packet headers alone.
- **L4** (95 features): round-trip time, bytes in flight, retransmissions, receive window. Requires tracking per-flow TCP state across packets.
- **L7** (55 features): chunk sizes and chunk inter-arrival times. Requires inferring chunk boundaries inside encrypted traffic.

The task is a **future-horizon binary prediction**: a yes/no question about a short time window ahead. At time t , using only information available so far, predict whether resolution drops during $(t, t+\Delta]$ (we sweep Δ from 10 to 30 seconds). Positive cases make up 1.7–3.9% of windows depending on the horizon. We will split the data by *session*, so that no session appears in both training and test data, and additionally report a temporal split—train on earlier sessions, test on later ones—to check for *drift* (the model degrading as conditions change over time).

Machine Learning

Baselines (simple comparison points). Tree-based classifiers—*random forest* and *gradient boosting*, which learn many if-then rules from examples—trained on the full feature set and on each fixed tier individually. Because a session already at the lowest resolution cannot downswitch, we also train a baseline using *only the current resolution* and report every result as improvement over it; otherwise a high score may reflect nothing but “high quality has room to fall.”

Adaptive method. A *cascade* (also called sequential acquisition): a classifier using only L3 features runs first and outputs a *probability* (how likely a downswitch is, from 0 to 1). If that probability sits near the *decision boundary*—the cutoff between “yes” and “no,” meaning the model is unsure—the system computes L4 features and reclassifies, and if still unsure, L7. Acquisition stops once *confidence* is high enough or the full feature set is reached. Because these thresholds are compared against probabilities, we *calibrate* each model so its outputs mean what they claim.

Evaluation

Prediction quality. We will report precision (of the windows we flagged, how many really degraded), recall (of the windows that degraded, how many we caught), F1, and a confusion matrix. Our headline metric is **PR-AUC**, which summarizes the precision–recall tradeoff across all cutoffs. Because downswitches are *rare* (*class imbalance*: under 4% of windows), plain accuracy and ROC-AUC both look strong even for a model that catches almost nothing. We report results per service as well as pooled, since services differ in how they adapt. We also measure *lead time*: how many seconds before a downswitch the model first flags the session.

Systems cost. We measure feature-extraction time for each tier empirically on `netflix.pcap`, then sweep the cascade’s confidence threshold to plot prediction quality against cost and identify *operating points*: settings that stay close to full-model performance at substantially lower cost. We additionally model each tier’s *time-to-availability* (L7 features cannot exist until chunks have arrived) in order to plot quality against effective warning time; we label this clearly as a cost model rather than a measurement.

Controls. The cascade is compared against each fixed tier and against *random* escalation at a matched rate. Without that last comparison, a cascade that appears to win may only be showing that more features sometimes help. If a single fixed tier turns out to dominate the adaptive approach on both axes, we report that as our finding.

Learning Objective

We expect to learn how feature representation and selection affect both model quality and systems cost, and to practice designing an adaptive pipeline that spends compute only on hard examples. More broadly, the project is about a deployment tradeoff in real-time ML: maximizing accuracy is not the only objective when latency (how fast you must answer) and feature cost also matter.